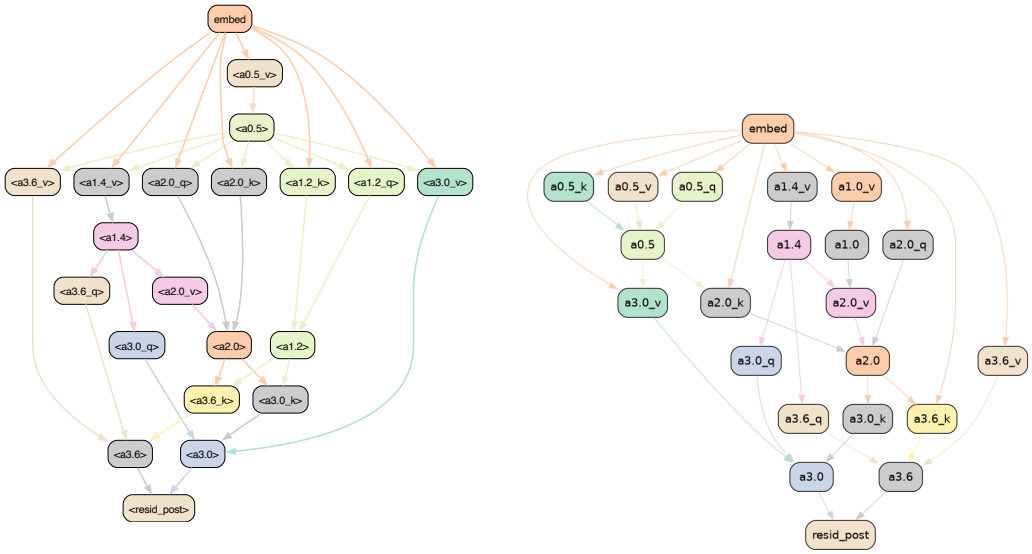


(a) Canonical Docstring circuit (37 edges)

(b) ACDC circuit (KL-divergence, $\tau = 0.095$)

Figure 18: Our implementation of the Docstring circuit (Heimersheim and Janiak, 2023), compared to an ACDC-generated circuit.

Metric	**Full model**	**ACDC KL** $\tau = 0.005$	**ACDC KL** $\tau = 0.095$ (Fig. 18b)	**ACDC LD** $\tau = 0.067$ (Fig. 19)	**Manual** 8 heads, all connections	**Ground-truth circuit** (Fig. 18a)
KL-divergence	0	0.33	1.2	0.67	0.83	1.1
Mean logit diff.	0.48	0.58	-1.7	0.32	-0.62	-1.6
Num. of edges	1377	258	34	98	464	37

Table 4: Comparing our ACDC docstring results to the ground-truth from Heimersheim and Janiak (2023) using their metrics. We compare (from left to right) the full model, the subgraph from ACDC runs optimizing for KL divergence ($\tau = 0.005$ and 0.095) and logit difference ($\tau = 0.067$), as well as the two subgraphs made manually from Heimersheim and Janiak (2023): One including all connections between the given attention heads, and one using only the given circuit. The metrics used are KL divergence between full-circuit outputs and resamble-ablated output (lower is better), mean logit difference between correct and wrong completions (higher is better), and the number of edges in the circuit (lower is better).

includes the edge connections that the authors speculated were most important but did not evaluate due to a lack of software for edge-editing. We find (a) overlapping heads, (b) heads found by ACDC only, and (c) heads found in the manual interpretation only. In the first class (a) we find heads 0.5, 1.4, 2.0, 3.0, and 3.6. All these manually identified heads are recovered by ACDC. In class (b) we find head 1.0 which the authors later add to their circuit to improve performance; ACDC shows for the first time where this head belongs in the circuit. In class (c) we find heads 0.2, 0.4 and 1.2. However, the first two of these are not actually relevant under the docstring distribution and only added by the authors manually. Head 1.2 is considered a non-essential but supporting head by the authors and not identified by ACDC at the chosen threshold of $\tau = 0.095$ (for KL divergence). This might be because 1.2 is less important than the other heads, and indeed we recover this head in larger subgraphs (such as the subgraph in Figure 19).

We compare the numerical results between the ACDC circuits and the circuit described in Heimersheim and Janiak (2023) in Table 4. In addition to the $\tau = 0.095$ run (Figure 18b) we perform a run with lower KL divergence threshold of $\tau = 0.005$ recovering a larger circuit (258 edges) containing also head 1.2 that was missing earlier.

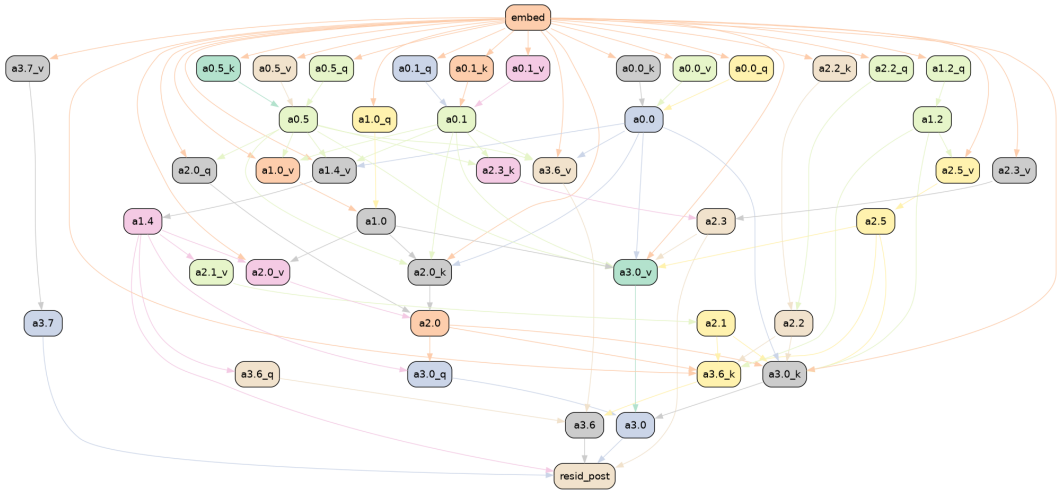


Figure 19: ACDC-found subgraph for docstring task minimizing logit difference ($\tau = 0.067$) instead of KL divergence.

Since Heimersheim and Janiak (2023) use logit difference as their metric, we add an ACDC run that optimizes logit difference rather than KL divergence (see Appendix C for details on this adjustment) with threshold $\tau = 0.067$. This circuit (Figure 19) recovers the relevant manual-interpretation heads (including 1.2) as well.[10] It is even more specific, containing 93% less edges than the full circuit. This is also 79% less edges than the head-based circuit from Heimersheim and Janiak (2023) while achieving a better score on all metrics.

Note that there are two versions of the manual circuit we consider. There is (i) the set of 8 heads given in Heimersheim and Janiak (2023) that the authors test with a simple methods (not specifying edges), and (ii) the circuit of 39 edges as suggested by the authors that they were not able to test due to not having software to implement editable transformer computational graphs in PyTorch. We reconstruct this circuit, shown in Figure 18a, from their descriptions and perform tests (Table 4).

In case (i) the ACDC run (threshold $\tau = 0.005$) achieves better performance in both metrics, Logit Difference and KL divergence, while being more specific (258 edges) when compared to the set of heads found by Heimersheim and Janiak (2023). In the more specific case (ii) the ACDC run (with threshold $\tau = 0.095$) closely matches the manual interpretation, with a very similar circuit recovered (Figure 18). The ACDC run is slightly more specific but has slightly worse KL divergence and Logit Difference.

A limitation worth noting is that we applied ACDC to a computational graph of attention heads and their query, key and value computational nodes, while Heimersheim and Janiak (2023) considered the attention heads outputs into every token position separately. This allowed them to distinguish two functions fulfilled by the same attention head (layer 1, head 4) at different positions, which cannot be inferred from the output of ACDC alone at any level of abstraction (Section 2.2) we studied in this work. We make this choice for performance reasons (the long sequence length would have made the experiments significantly slower) but this is not a fundamental limitation. In Appendix K we use ACDC to isolate the effects of individual positions in a different task.

H.2 Additional docstring experiments

Logit difference metric: To compare ACDC more closely with the docstring work (Heimersheim and Janiak, 2023), we added an ACDC run with the objective to maximize the logit difference metric. We used a threshold of $\tau = 0.067$ and found the subgraph shown in Figure 19. We found that ACDC performed better than SP and HISP when using the logit difference metric (Figure 7).

Zero activations: Unlike in the case of induction (Section 4.2), we found that using zero activations rather than random (corrupted) activations, lead to far worse results. For example, with $\tau = 0.067$

[10] Again, not considering heads 0.2 and 0.4 which are not actually relevant under the docstring distribution.

(the same threshold that generated Figure 19 except with zero activations) we get a circuit with 177 edges (Figure 19 has 98), as well as a KL divergence of 3.35 and a logit difference of -2.895. All these metrics are worse than the subgraphs generated with corrupted activations (Table 4).

I Tracr tasks: details and qualitative evidence

In this Appendix we discuss the two tracr tasks we studied in Section 4, as well as additional experiments that studied ACDC when applied at a neuron level.

We used a transformer identical to the one studied in Lindner et al. (2023), and refer to that work for details on the tracr-xproportion task (called the `frac_prevs` task in their paper). We also studied the tracr-reverse task, described in the tracr Github repository.[11]

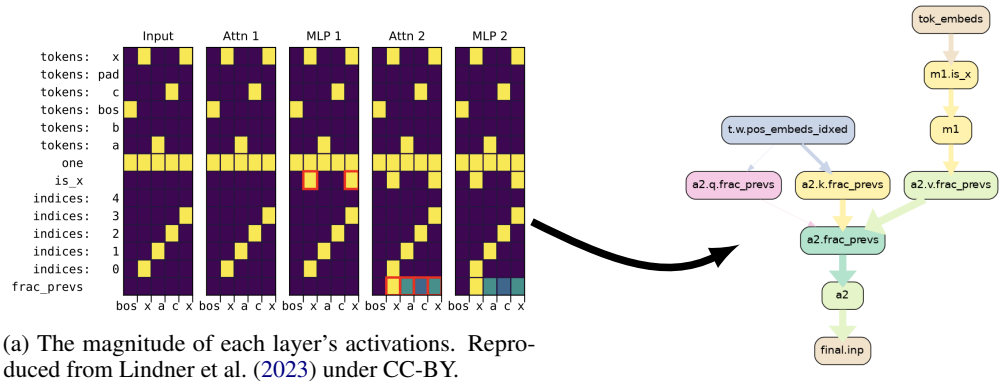
We make one modification to the traditional ACDC setup. We set the positional embeddings equal to randomized positional embeddings in the corrupted datapoints — otherwise, we don't recover any of the circuit components that depend only on positional embeddings (and not token embeddings). We describe the two tasks that we studied in the main text and describe futher results that broke these computational graphs down into neurons.

I.1 tracr-xproportion

We used the proportion task from the tracr main text, and used as metric the L2 distance between the correct list of proportions and the recovered list of proportions. For the corrupted dataset, we let $(x'_i)_{i=1}^n$ be a random permuation of $(x_i)_{i=1}^n$ with no fixed points.

When we ran ACDC at the neuron level, as shown in Figure 20b, there are no extra nodes present that were not used by this tracr model. In fact, this computational graph visualization produced by ACDC is more compact than the complete view of the states of the residual stream which is illustrated in Figure 20a (from Lindner et al. (2023)). In this case, the transformer is small enough for a practitioner to study each individual state in the forward pass. However, for larger transformers this would be intractable, which necessitates the use of different interpretability tools such as ACDC.

See Fig. 21a for the full circuit (without decomposition into residual stream dimensions).



(a) The magnitude of each layer's activations. Reproduced from Lindner et al. (2023) under CC-BY.

(b) ACDC visualization.

Figure 20: Two visualizations of how a `tracr`-compiled transformer completes the `frac_prevs` task. The ACDC circuit is specific to the individual neurons and residual stream components. This is more fine-grained than the ground truth we use throughout the work. This experiment was coded in `rust_circuit` and is not reproducible using the Transformer Lens code yet.

I.2 tracr-reverse

To test ACDC on more than one example of a tracr program, we also used the 3-layer transformer that can reverse lists (the tracr-reverse task). Once more, the outputs of the transformer are not

[11] URL: https://github.com/deepmind/tracr, file: `README.md`